Week 3, Day 4: Sagas, Outbox, Idempotency, CQRS and Binary Search Trees

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topic: distributed transactions, saga (choreography and orchestration), outbox pattern, CQRS, idempotency | DSA: Binary search trees

Today's Schedule

- **0:00 to 0:50** Study why distributed transactions are hard, then sagas (sections 1 and 2).
- **0:50 to 1:30** Study the dual-write problem and the outbox and inbox patterns (section 3).
- **1:30 to 2:00** Study idempotency (section 4).
- **2:00 to 2:30** Study CQRS and event sourcing (section 5), then the worked example (section 6).
- **2:30 to 3:30** DSA: solve the 3 BST problems (section 8). Spend at most 20 minutes per problem before reading the hint.
- **3:30 to 4:00** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (60 to 90 min):** do the hands-on exercise in section 7.

1. Why Distributed Transactions Are Hard

- In a monolith, one database transaction can update orders, payments and stock atomically. With a **database per service**, no single ACID transaction spans them.
- **Two-phase commit (2PC / XA)** can coordinate several resources, but it is rarely used between microservices because it **blocks** resources while waiting for the coordinator, the coordinator is a single point of failure, latency is high, availability drops, and many brokers and NoSQL stores do not support it.
- The usual answer is to accept **eventual consistency**: each service commits its own local transaction, and the system converges to a consistent state through messages. When something fails halfway, you **compensate** instead of rolling back.
- Two problems to solve together: **coordinating a multi-step business process** (saga) and **reliably publishing events when data changes** (outbox). A third, **handling duplicate messages**, is idempotency.

2. The Saga Pattern

Definition: a saga is a sequence of **local transactions**, one per service. Each step commits locally and triggers the next one by event or command. If a step fails, the saga runs **compensating transactions** for the steps that already succeeded.

Compensation is a semantic undo, not a rollback. The earlier step really committed. For example, a payment is refunded, a reservation is released, an order is marked CANCELLED. Compensations must themselves be **idempotent and retryable**.

Example: place an order

1. Order service creates the order in state PENDING.
2. Payment service charges the customer.
3. Inventory service reserves stock.
4. If all succeed, the order becomes CONFIRMED.
5. If inventory fails, payment is **refunded** (compensation) and the order becomes CANCELLED.

Choreography

- There is **no central coordinator**. Each service listens for events and publishes its own. OrderCreated triggers payment, PaymentCompleted triggers inventory, StockFailed triggers a refund.
- **Pros:** simple for a few steps, loosely coupled, no extra component.
- **Cons:** the overall flow is spread across many services and hard to see, easy to create cyclic dependencies, and hard to track or debug as steps grow.

Orchestration

- A central **orchestrator** (a dedicated saga service, or a workflow engine such as Temporal, Camunda, AWS Step Functions, Netflix Conductor or Axon) tells each participant what to do and handles replies, timeouts and compensation.
- **Pros:** the flow is written in one place, easy to understand, monitor and change, with built-in state and timeout handling.
- **Cons:** an extra component to run, and a risk that the orchestrator becomes a god service. Keep business rules inside participants and only the **sequence** in the orchestrator.

How to choose: choreography for short, simple flows (two to four steps). Orchestration once the flow has several steps, branches, timeouts or a need for visibility.

Hard parts of sagas

- **No isolation:** other transactions can see intermediate states (a PENDING order, a reserved but unconfirmed stock). Countermeasures: **semantic locks** (explicit PENDING states that other operations respect), commutative updates, and re-reading values before acting.
- **Order the steps well:** put steps that can be compensated first, a **pivot step** (the point of no return) next, and steps that cannot fail or can be retried until they succeed last. Things that cannot be undone, such as sending an email, belong at the end.
- **Failures and timeouts:** record saga state, retry transient errors, use dead letter queues, and flag stuck sagas for manual handling. Monitor sagas that stay PENDING too long.
- **Idempotent handlers:** every step may receive the same message more than once.

3. The Dual-Write Problem and the Outbox Pattern

The problem: after saving an order you also want to publish OrderCreated to Kafka. These are two different systems, so there is no shared transaction.

- Commit the database first, then publish: if the service crashes in between, **the event is lost**, and downstream services never learn about the order.
- Publish first, then commit: if the commit fails, you have announced an **order that does not exist**.

Transactional outbox

1. In the **same local database transaction**, write the business row **and** a row into an **outbox table** describing the event.
2. A separate **relay** reads unpublished outbox rows and publishes them to the broker, then marks them as published.
3. Because the business change and the event are saved atomically, you can never have one without the other.

```
CREATE TABLE outbox (
    id             uuid PRIMARY KEY,
    aggregate_type text        NOT NULL,
    aggregate_id   text        NOT NULL,
    event_type     text        NOT NULL,
    payload        jsonb       NOT NULL,
    created_at     timestamptz NOT NULL DEFAULT now(),
    published_at   timestamptz
);
CREATE INDEX idx_outbox_unpublished ON outbox (created_at) WHERE published_at
IS NULL;
```

```
@Transactional
public Order placeOrder(PlaceOrderRequest req) {
    Order order = orders.save(Order.pending(req));
    outbox.save(OutboxEvent.of("Order", order.getId().toString(),
                               "OrderCreated", toJson(order)));
    return order;                    // both rows commit together, or
neither does
}
```

Two ways to relay

- **Polling publisher:** a scheduled job selects a batch of unpublished rows, sends them, and marks them published. With several instances, use SELECT ... FOR UPDATE **SKIP LOCKED** (Week 2, Day 5) so instances do not process the same rows. Simple, with some latency and load on the database.
- **Change Data Capture (CDC):** a tool such as **Debezium** reads the database transaction log (the write-ahead log in PostgreSQL) and publishes outbox rows to Kafka, with an outbox event router. Low latency and no polling load, but more infrastructure to run.

```
SELECT id, aggregate_id, event_type, payload
FROM outbox
WHERE published_at IS NULL
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Guarantees and details

- The relay gives **at-least-once** delivery: a crash after sending but before marking published causes a duplicate. So **consumers must be idempotent**.
- Use the **aggregate id as the Kafka message key**, so events for one order stay in order within a partition.
- **Clean up** old published rows with a scheduled job or partitioning.
- Include an **event id** and a **version** in each event, so consumers can deduplicate and handle schema changes.
- Publishing from @TransactionalEventListener(AFTER_COMMIT) is better than publishing inside the transaction, but it can still lose the event if the service crashes right after the commit. The outbox closes that gap.

Inbox pattern (consumer side)

- The consumer stores the id of every processed message in a table, in the **same transaction** as its business changes. A unique constraint on the message id makes duplicates harmless.

4. Idempotency

Definition: doing an operation several times has the same effect as doing it once. It is essential because networks, brokers and retries mean duplicates **will** happen.

Techniques

- **Naturally idempotent operations:** set a value rather than increment it, upsert instead of insert, mark an order CONFIRMED (a second time changes nothing).
- **Idempotent consumer:** record each processed message id with a **unique constraint**. If the insert finds a duplicate, skip the work.
- **Idempotency key on APIs:** the client sends a unique key (a UUID) with a POST. The server stores the key with the result, and a repeated key returns the **stored result** instead of acting again. Handle two simultaneous requests with the same key, by having the first one record the key as in progress and others wait or receive 409. Expire old keys after a retention window.
- **Optimistic locking and conditional updates** prevent applying the same change twice, for example UPDATE … WHERE status = 'PENDING'.
- **At-least-once delivery plus idempotent processing gives effectively-once results.** Kafka's exactly-once features cover reads and writes within Kafka, but they do not make your database updates or HTTP calls idempotent.

```
INSERT INTO processed_message (message_id, handled_at)
VALUES (:messageId, now())
ON CONFLICT (message_id) DO NOTHING;
-- if the update count is 0, the message was already processed: skip it
```

```
@Transactional
public void on(PaymentCompleted event) {
    int inserted = processed.insertIfNew(event.messageId());   // the SQL
above
    if (inserted == 0) return;                                  // duplicate:
ignore
    inventory.reserve(event.orderId(), event.items());          // same
transaction
}
```

5. CQRS and Event Sourcing

CQRS (Command Query Responsibility Segregation): separate the model that **changes** data (commands) from the model that **reads** it (queries).

- **Levels:** at the simplest level you just use separate classes and DTOs for writes and reads, over the same database. At the advanced level the read side is a **separate store** (denormalized tables, Elasticsearch, Redis) kept up to date by events from the write side.
- **Why:** reads and writes often have very different shapes and loads. A read model can be shaped exactly for a screen, scaled separately, and can **combine data from several services**, which is how you answer queries that would otherwise need cross-service joins.
- **Costs:** more moving parts, **eventual consistency** between the write and read sides (a user may not see their change immediately), duplicated data and projection code.
- **Use it** when read and write needs clearly differ or you need cross-service views. **Do not apply it everywhere.** A simple CRUD service does not need it.
- **Alternative for cross-service queries: API composition**, where an aggregator service calls several services and merges the results. Simpler, but slower and dependent on all of them being up.

Event sourcing

- Store the **sequence of events** as the source of truth, instead of just the current state. The current state is rebuilt by replaying events, and **snapshots** speed that up.
- **Pros:** a complete audit trail, the ability to rebuild read models and to reason about history.
- **Cons:** a different way of thinking, harder queries (so it is usually paired with CQRS), schema evolution of old events, and operational complexity.
- It is **independent of CQRS**. You can use either one without the other. Mention it, but be honest that most business systems do not need it.

6. Worked Example: Order Flow with Saga, Outbox and Idempotency

1. **Order service:** in one transaction, create the order as PENDING and write an OrderCreated row to its outbox. The relay publishes it to Kafka with the order id as the key.
2. **Payment service:** consumes OrderCreated using an idempotent consumer (dedupe by message id), charges the customer using the order id as an idempotency key towards the payment provider, then writes PaymentCompleted or PaymentFailed to **its own outbox**.
3. **Inventory service:** consumes PaymentCompleted, reserves stock, and writes StockReserved or StockFailed to its outbox.

4. **Success path:** Order service consumes StockReserved and sets the order to CONFIRMED.
5. **Failure path:** on StockFailed, Payment service **refunds** (compensation, itself idempotent) and emits PaymentRefunded. Order service sets the order to CANCELLED. On PaymentFailed, the order is cancelled directly.
6. **Safety nets:** retries with backoff, a dead letter topic for poison messages, a timeout that flags orders stuck in PENDING, and dashboards showing the saga state.

This combination (saga for coordination, outbox for reliable publishing, idempotent consumers for duplicates) is the standard answer to the question "how do you handle transactions across microservices".

7. Optional Hands-On (60 to 90 Minutes)

1. In order-service (PostgreSQL), add the outbox table and an OutboxEvent entity. Change placeOrder to write the order and the outbox row in one transaction.
2. Add a polling relay with @Scheduled that selects a batch with FOR UPDATE SKIP LOCKED, **logs the event** (a stand-in for Kafka, which comes tomorrow), and sets published_at.
3. Prove atomicity: throw an exception after the outbox insert and confirm that **neither** row is saved.
4. In a second service (or a second consumer class), add a processed_message table and the idempotent handler. Deliver the **same message twice** in a test and assert that the effect happens once.
5. Sketch the order saga on paper with the compensation for each step, and write down the order status values.
6. Next week, replace the log statement with a real Kafka producer, so the outbox is ready for the mini project.

8. DSA: Binary Search Trees (3 Problems)

BST property: for every node, all values in the left subtree are smaller and all values in the right subtree are larger. An **inorder traversal of a BST gives sorted order**, which is the key to many problems. Search, insert and delete take O(h), which is O(log n) when balanced and O(n) when skewed. These problems use the TreeNode class from Day 3.

Problem 1: Validate Binary Search Tree

- **Idea:** pass an allowed **range (lower and upper bound)** down the recursion. Going left, the current value becomes the new upper bound. Going right, it becomes the new lower bound. A node outside its range makes the tree invalid.
- **Common bug:** comparing a node only with its immediate children. That misses a deeper node that violates an ancestor's bound.
- **Complexity:** O(n) time, O(h) space.
- **Edge cases:** node values equal to Integer.MIN_VALUE or MAX_VALUE (use null bounds or long), duplicate values (strictly invalid here), a single node or empty tree.
- **Alternative:** do an inorder traversal and check that each value is greater than the previous one.

```
public boolean isValidBST(TreeNode root) {
    return valid(root, null, null);
}

private boolean valid(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
    if (low != null && node.val <= low) return false;
    if (high != null && node.val >= high) return false;
    return valid(node.left, low, node.val) && valid(node.right, node.val,
high);
}
```

Problem 2: Kth Smallest Element in a BST

- **Idea:** do an **iterative inorder traversal** with an explicit stack and stop when you have visited k nodes. Inorder visits values in ascending order.
- **Complexity:** O(h + k) time, O(h) space.
- **Edge cases:** k equal to 1 (the leftmost node), k equal to the number of nodes.
- **Follow-up to prepare:** if the tree is modified often and you query often, store the **size of each subtree** in the nodes. Then you can find the kth element in O(h) by comparing k with the left subtree size.

```
public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode cur = root;
    while (cur != null || !stack.isEmpty()) {
        while (cur != null) {
            stack.push(cur);
            cur = cur.left;
        }
        cur = stack.pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
    return -1;                                  // k is larger than the tree
size
}
```

Problem 3: Lowest Common Ancestor of a Binary Search Tree

- **Idea:** use the BST ordering. If both p and q are smaller than the current node, go left. If both are larger, go right. Otherwise the current node **splits** them (or equals one of them), so it is the lowest common ancestor.
- **Complexity:** O(h) time, O(1) space for the iterative version.
- **Edge cases:** p is an ancestor of q (the answer is p), p and q are the same node.
- **Follow-up:** the general binary tree version (no ordering) uses a recursive search that returns a node when it finds p or q, and the first node that sees results from both sides is the answer. It is O(n).

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode node = root;
    while (node != null) {
        if (p.val < node.val && q.val < node.val) node = node.left;
        else if (p.val > node.val && q.val > node.val) node = node.right;
        else return node;
    }
    return null;
}
```

9. Interview Questions to Answer Aloud

1. Why can you not use a normal database transaction across microservices? Why is 2PC usually avoided?

2. What is a saga? What is a compensating transaction, and how is it different from a rollback?
3. Choreography vs orchestration: pros, cons and when you would choose each.
4. What isolation problems do sagas have, and how do semantic locks help?
5. What is the dual-write problem?
6. How does the transactional outbox pattern work? Compare a polling publisher with CDC (Debezium).
7. What delivery guarantee does the outbox give, and what must consumers therefore do?
8. What is the inbox pattern, and how does a unique constraint help?
9. What does idempotent mean? How do you make a POST endpoint and a message consumer idempotent?
10. Does Kafka's exactly-once feature remove the need for idempotent consumers? Why or why not?
11. What is CQRS, and what are its costs? When would you not use it?
12. What is event sourcing, and how does it relate to CQRS?
13. Walk me through the order saga when stock reservation fails after payment succeeded.
14. How do you find and handle sagas that are stuck?

10. End-of-Day Checklist

- [ ] I can explain why distributed transactions are hard and why 2PC is avoided.
- [ ] I can describe a saga with both styles, plus the compensation for each step.
- [ ] I can explain the dual-write problem and the outbox pattern, including the relay options.
- [ ] I can design an idempotent consumer and an idempotency key for an API.
- [ ] I can explain CQRS and event sourcing, and when not to use them.
- [ ] I solved all 3 BST problems and can explain the range-bound idea and inorder property.
- [ ] (Optional) I built the outbox table and proved atomicity with a failing transaction.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 3, Day 5): Kafka fundamentals (topics, partitions, brokers, producers, consumers, consumer groups), plus more tree problems.